

Relatório técnico da linguagem MemeScript

Trabalho Prático 1 de Linguagens Formais e Compiladores

UNIJUÍ • Professor Marcos Ronaldo Melo Cavaleiro

Integrantes: Cristian dos Santos Siqueira, Marco Antônio Hendges, Pedro Rockenbach Frosi e William Rafael Fagundes • Versão 1.0 • Outubro de 2026

Objetivo e escopo

A MemeScript é uma linguagem temática baseada em memes, com tipos explícitos e transpilação para Python. Seu transpilador reconhece o código-fonte, produz tokens, constrói uma Árvore Sintática Abstrata (AST), verifica regras semânticas e gera um programa equivalente. A geração ocorre somente após a validação completa.

A versão implementada oferece três tipos primitivos, declaração e atribuição, aritmética com precedência, comparações, decisão com alternativa, repetição, entrada e saída. Um comando adicional permite interromper o laço mais próximo. A implementação usa apenas a biblioteca padrão do Python.

Três decisões próprias de projeto

A BIRL-Lite do enunciado serviu apenas como modelo do nível de formalização. As três decisões abaixo mudam a forma da gramática, e não só os nomes das palavras reservadas.

1. Atribuição introduzida por palavra reservada. Na BIRL-Lite, uma atribuição é `id = expressão`; Na MemeScript, é `RECEBA id = expressão`. Com isso, todo comando começa com uma palavra reservada própria (RECEBA, QUERO_CAFE, AMOSTRADINHO, PODE_ISSO_ARNALDO, BORA_BILL ou DESCANSAR_NE), e o parser escolhe a produção de statement olhando apenas o primeiro token, sem precisar fatorar a gramática. Atribuição e declaração também ficam visualmente distintas.

2. Seção de declarações separada e inicialização obrigatória. Na BIRL-Lite, declarações e comandos se misturam no mesmo bloco e o valor inicial é opcional. Na MemeScript, todas as declarações vêm no início do programa, antes do primeiro comando, e cada uma exige inicializador, como em `SABOR_INTEIRO x = 0`. Assim nunca existe variável sem valor, e a tabela de símbolos está completa antes da análise dos comandos. Uma declaração no meio dos comandos é erro sintático.

3. Terminadores distintos para programa e blocos. A BIRL-Lite usa a mesma palavra BIRL para fechar condicionais, laços e o próprio programa. A MemeScript usa JA_ACABOU_JESSICA apenas para o programa e ATA apenas para blocos, e um único ATA fecha o condicional inteiro, incluindo o ramo ERROU. Um ATA esquecido é apontado com precisão (encontrado JA_ACABOU_JESSICA; esperado ATA), e a ambiguidade do senão pendente não pode ocorrer.

Outras diferenças em relação ao modelo: AMOSTRADINHO aceita várias expressões separadas por vírgula, existe o tipo texto, e DESCANSAR_NE interrompe o laço mais próximo, com uma regra semântica que o proíbe fora de BORA_BILL.

Aspecto	BIRL-Lite	MemeScript
Atribuição	<code>x = e;</code>	<code>RECEBA x = e;</code>
Declarações	Misturadas aos comandos, inicializador opcional	Seção no início, inicializador obrigatório

Aspecto	BIRL-Lite	MemeScript
Fechamento	BIRL fecha blocos e programa	ATA fecha blocos; JA_ACABOU_JESSICA fecha o programa
Saída	Uma string ou expressão	Uma ou mais expressões
Tipos	Inteiro e real	Inteiro, real e texto
Laço	Apenas a condição	Condição e interrupção com DESCANSAR_NE

Justificativa da linguagem destino

A linguagem destino é Python 3.10 ou superior. A escolha foi informada ao professor no primeiro marco e se apoia em três motivos técnicos:

1. Todas as construções da MemeScript têm equivalente direto no destino: atribuição, if/else, while, break, input() e print().
2. O sistema de tipos é inteiramente da MemeScript. Python só verifica tipos durante a execução, então nenhuma checagem é delegada ao destino: uso antes da declaração, redeclaração e incompatibilidades são detectados pelo transpilador antes de qualquer código ser gerado.
3. Os tipos continuam guiando a geração: variáveis reais recebem float(...), a leitura usa int(input()) ou float(input()) conforme o tipo declarado, a divisão produz sempre real e comparações são exibidas como verdadeiro ou falso.

Como Python delimita blocos por indentação, o gerador calcula a indentação a partir da profundidade de cada nó na AST, e não do texto do código-fonte.

Vocabulário e tipos

Os significados abaixo são decisões da linguagem. O uso do prefixo SABOR adapta o bordão ao sistema de tipos e não corresponde a um significado técnico do meme original. O número 67 permanece um literal inteiro comum.

Palavra reservada	Papel na linguagem
E_HORA_DO_SHOW	Delimita o início do programa
JA_ACABOU_JESSICA	Delimita o fim do programa
SABOR_INTEIRO	Declara variável inteira
SABOR_REAL	Declara variável real
SABOR_TEXTO	Declara variável de texto
RECEBA	Atribui a uma variável já declarada
QUERO_CAFE	Lê um valor conforme o tipo da variável
AMOSTRADINHO	Exibe uma ou mais expressões
PODE_ISSO_ARNALDO	Abre um condicional
ERROU	Introduz a alternativa do condicional
BORA_BILL	Abre um laço controlado por condição
DESCANSAR_NE	Interrompe o laço mais próximo
ATA	Fecha um condicional inteiro ou um laço

Tipo	Literal de exemplo	Representação no destino
inteiro	67	int
real	6.7	float
texto	"Receba!"	str

Comparações produzem o tipo interno lógico, representado por bool no destino e exibido como verdadeiro ou falso. Ele não possui palavra reservada de declaração e pode aparecer em condições, igualdade entre resultados lógicos e saída. Um inteiro não vale implicitamente como condição.

Alfabeto e reconhecimento léxico

O alfabeto Σ da linguagem é o conjunto dos caracteres que podem aparecer no código fora de textos e comentários:

$$\Sigma = \{ A..Z, a..z, 0..9, _ , \\ + \ - \ * \ / \ = \ ! \ < \ > , \\ (\) \ , \ ; \ . \ " , \\ \text{espaço}, \text{tab}, \text{CR}, \text{LF} \}$$

Dentro de um literal de texto ou de um comentário, aceita-se qualquer caractere Unicode codificado em UTF-8, exceto quebra de linha e, nos textos, os controles U+0000 a U+001F, aspas e barra invertida sem escape. Um caractere fora de Σ no código, como @, #, ç ou letra acentuada, gera erro léxico com linha e coluna. Os caracteres ! e . pertencem a Σ , mas só formam token dentro de != e de um número real; isolados, também são erro léxico. Identificadores e palavras reservadas usam apenas ASCII.

O leitor de arquivos aceita UTF-8 com ou sem BOM. Posições usam linhas e colunas iniciadas em 1. Um tab conta como um caractere na coluna, sem expandir para uma largura visual fixa. CRLF conta como uma única quebra de linha; CR e LF isolados também encerram linhas.

Tabela de tokens por padrão

As ERs abaixo usam a sintaxe do módulo re do Python. INTEGER, REAL, STRING e IDENTIFIER são categorias emitidas. Palavras reservadas, operadores e delimitadores recebem como tipo do token a própria grafia, por exemplo RECEBA ou ==.

Categoria	Expressão regular ou padrão	Exemplo	Descrição
INTEGER	<code>[0-9]+</code>	67	Inteiro decimal sem sinal
REAL	<code>[0-9]+\.[0-9]+</code>	6.7	Real decimal sem sinal
STRING	<code>"(?:[^\x00-\x1f] \\["\\\nrt])*"</code>	"café"	Texto entre aspas duplas
IDENTIFIER	<code>[A-Za-z_][A-Za-z0-9_]*</code>	energia	Nome não reservado
WHITESPACE	<code>[\t\r\n]+</code>	espaço	Ignorado, atualiza posição
COMMENT	<code>//^[^r\n]*</code>	// comentário	Ignorado até o fim da linha

WHITESPACE e COMMENT são categorias de reconhecimento descartadas, não terminais da gramática. O token EOF é um marcador de fim da sequência e não corresponde a caracteres do código-fonte.

Sinais não integram o token numérico: -67 corresponde aos tokens - e INTEGER. Não há notação científica, vírgula decimal nem formas .5 ou 5. para reais. Strings não admitem quebra de linha literal ou controles U+0000 a U+001F. Aceitam os escapes de nova linha, retorno, tab, aspas e barra invertida: \n, \r, \t, \", \\.

Palavras reservadas e símbolos

Cada uma das treze palavras da tabela de vocabulário é um padrão literal completo. O lexer primeiro reconhece um IDENTIFIER e consulta o dicionário de reservadas. Assim, ATA é palavra reservada, ATA_2 é identificador e ata é outro identificador.

Tokens emitidos	Padrão	Exemplo	Função
E_HORA_DO_SHOW até ATA	Literal exato do vocabulário	RECEBA	Treze reservadas
+ e -	[+-]	+	Soma, subtração ou sinal unário
* e /	[*/]	*	Multiplicação e divisão
=	=	=	Inicialização e atribuição
== e !=	== ou !=	==	Igualdade e diferença
< e <=	< ou <=	<=	Comparação de ordem
> e >=	> ou >=	>=	Comparação de ordem
(e)	[()]	(	Agrupamento e argumentos
,	,	,	Separação de expressões na saída
;	;	;	Fim de comando simples
EOF	Marcador acrescentado pelo lexer	fim	Fim da sequência de tokens

Maior casamento

Na posição atual, o lexer testa todos os padrões e escolhe o lexema mais longo. Em empate, usa a ordem da lista PATTERNS. No padrão de operadores, as alternativas de dois caracteres vêm antes das de um caractere. Portanto, == forma um único token, e não dois tokens =. O comentário // também ganha do operador / pelo maior comprimento.

O analisador avança apenas após reconhecer ou descartar o lexema. Se nenhum padrão casar, informa erro léxico com linha e coluna. Texto sem fechamento, escape não suportado e @ são exemplos. Literais reais infinitos e inteiros acima do limite de conversão do runtime também são rejeitados pelo lexer.

As ERs reconhecem linguagens regulares. Aninhamento de parênteses, if e while depende da gramática e do parser, não do reconhecimento de um token isolado.

Definição formal da gramática

A gramática livre de contexto é $G = (V, T, P, S)$. $S = \text{program}$. P é o conjunto de produções EBNF apresentado na próxima página. Chaves indicam zero ou mais repetições, colchetes indicam uma parte opcional e a barra vertical indica alternativa. Aspas delimitam terminais literais.

```
V = { program, declaration, type, block, statement,
      assignment, input, output, if_stmt, while_stmt,
      break_stmt, expression, relop, additive,
      multiplicative, unary, primary }

T = { E_HORA_DO_SHOW, JA_ACABOU_JESSICA, SABOR_INTEIRO,
      SABOR_REAL, SABOR_TEXTO, RECEBA, QUERO_CAFE,
      AMOSTRADINHO, PODE_ISSO_ARNALDO,
      ERROU, BORA_BILL, DESCANSAR_NE, ATA,
      IDENTIFIER, INTEGER, REAL, STRING,
      "=", "+", "-", "*", "/", "==", "!=",
      "<", "<=", ">", ">=", "(", ")", ",", ";" }
```

Os elementos IDENTIFIER, INTEGER, REAL e STRING representam categorias léxicas com atributo de valor. Os nomes das palavras reservadas no conjunto T representam seus lexemas exatos. A gramática opera sobre a sequência de tokens; espaços e comentários já foram removidos. Após reconhecer program, o parser exige EOF para rejeitar conteúdo adicional.

Precedência e associatividade

Da maior para a menor precedência: agrupamento, sinais unários, multiplicação/divisão, soma/subtração e comparação. Os operadores aritméticos binários associam à esquerda: $10 - 3 - 2$ equivale a $(10 - 3) - 2$. Os sinais unários aninham recursivamente.

A produção expression aceita no máximo uma comparação por nível. A forma $1 < x < 10$ é rejeitada. Resultados de comparações entre parênteses podem aparecer em outras expressões, mas a análise semântica decide se a operação com eles é válida.

Os blocos têm fechamento explícito. O primeiro ATA disponível fecha a construção aberta mais interna. Em um if, ERROU pertence ao mesmo if, e um único ATA fecha seus dois ramos. Essa estrutura elimina a associação ambígua de else.

Produções completas em EBNF

```

program = "E_HORA_DO_SHOW", { declaration },
        { statement }, "JA_ACABOU_JESSICA" ;

declaration = type, IDENTIFIER, "=", expression, ";" ;
type = "SABOR_INTEIRO" | "SABOR_REAL" | "SABOR_TEXTO" ;
block = { statement } ;
statement = assignment | input | output | if_stmt
          | while_stmt | break_stmt ;

assignment = "RECEBA", IDENTIFIER, "=", expression, ";" ;
input = "QUERO_CAFE", "(", IDENTIFIER, ")", ";" ;
output = "AMOSTRADINHO", "(", expression,
        { ",", expression }, ")", ";" ;
if_stmt = "PODE_ISSO_ARNALDO", "(", expression, ")",
        block, [ "ERROU", block ], "ATA" ;
while_stmt = "BORA_BILL", "(", expression, ")", block, "ATA" ;
break_stmt = "DESCANSAR_NE", ";" ;

expression = additive, [ relop, additive ] ;
relop = "==" | "!=" | "<" | "<=" | ">" | ">=" ;
additive = multiplicative, { ( "+" | "-" ), multiplicative } ;
multiplicative = unary, { ( "*" | "/" ), unary } ;
unary = ( "+" | "-" ), unary | primary ;
primary = INTEGER | REAL | STRING | IDENTIFIER
        | "(", expression, ")" ;

```

Não há recursão à esquerda. A alternativa de statement é escolhida pela palavra reservada inicial. Um tipo abre uma declaração apenas antes dos comandos do programa. Depois disso, ou dentro de um bloco, uma declaração causa erro sintático.

program e block podem conter zero comandos. O gerador emite pass para um corpo vazio, preservando sua validade em Python. if sem alternativa é permitido, embora o programa completo de demonstração use os dois ramos.

Regras semânticas

A análise semântica percorre a AST depois do parser. A tabela de símbolos é um dicionário que associa cada nome ao seu tipo. Como o escopo é global, qualquer segunda declaração do mesmo nome é duplicada. A sintaxe permite um nome sem verificar sua existência; a tabela realiza essa verificação em uma fase separada.

1. Toda variável deve ser declarada antes do uso, inclusive no inicializador de outra variável.
2. O nome declarado entra na tabela apenas depois de validar seu inicializador. Assim, `SABOR_INTEIRO x = x + 1;` é inválido.
3. A declaração exige inicialização. Portanto, não há variável declarada sem valor inicial nesta versão.
4. Declarações e atribuições aceitam o mesmo tipo ou promoção de inteiro para real. Real para inteiro e conversões entre número e texto são rejeitados.
5. Operadores aritméticos, inclusive sinais unários, aceitam apenas inteiro ou real. Texto não possui concatenação.
6. Condições de `if` e `while` devem ter tipo lógico. A análise visita todos os ramos e o corpo do laço.
7. `DESCANSAR_NE` exige profundidade de laço maior que zero. Um `if` dentro de um laço preserva essa profundidade.

Operação	Tipos aceitos	Tipo de resultado
<code>+</code> , <code>-</code> , <code>*</code>	Dois números	real se houver real, senão inteiro
<code>/</code>	Dois números	Sempre real
Unário <code>+</code> ou <code>-</code>	Um número	Tipo do operando
<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	Dois números	lógico
<code>==</code> , <code>!=</code>	Dois números ou dois textos ou dois lógicos	lógico
Atribuição	Mesmo tipo ou inteiro para real	Tipo da variável

O gerador aplica `float(...)` em inicializações e atribuições a variáveis reais. Isso mantém valores como `0.0` mesmo quando o inicializador é inteiro. Entradas usam `int(input())`, `float(input())` ou `input()`, conforme o tipo.

Erros dependentes de valores em execução, como divisão por zero ou entrada não numérica, permanecem sob responsabilidade do programa Python gerado. A análise de tipos não prevê os valores que o usuário digitará.

Parser e representação por AST

O parser consome tokens em descida recursiva. Seus laços nas funções `additive` e `multiplicative` constroem árvores associativas à esquerda. Cada nó guarda linha e coluna do elemento que o originou, permitindo localizar os erros semânticos.

Produção ou papel	Implementação
program	Parser.parse
declaration e type	Parser.declaration e TYPE_KEYWORDS
block	Parser.block
statement e comandos	Parser.statement
expression e relop	Parser.expression e RELATIONAL
additive e multiplicative	Funções de mesmo nome
unary e primary	Funções de mesmo nome

Os nós são `Program`, `VarDecl`, `Assign`, `Read`, `Print`, `If`, `While`, `Break`, `Literal`, `Variable`, `Unary` e `Binary`. Palavras de fechamento, vírgulas e ponto e vírgula não viram nós. Parênteses determinam a estrutura da expressão, mas não precisam de um nó próprio.

Para `2 + 3 * 4`, a estrutura principal é:

```
{
  "node": "Binary",
  "operator": "+",
  "left": {"node": "Literal", "value": 2},
  "right": {
    "node": "Binary",
    "operator": "*",
    "left": {"node": "Literal", "value": 3},
    "right": {"node": "Literal", "value": 4}
  }
}
```

Este recorte omite `type_name`, `line` e `column` para destacar o agrupamento. O comando `--ast` exibe a representação completa. O nó `Print` contém essa expressão como um de seus argumentos.

Geração de Python

O gerador recebe a AST validada e a tabela de símbolos. Cada nó possui uma tradução definida. A profundidade do bloco determina a indentação. Expressões binárias e unárias recebem parênteses explícitos no destino, preservando a estrutura construída pelo parser.

Nó	Código gerado
VarDecl e Assign	Atribuição, com float para destino real
Read	input com conversão numérica quando necessária
Print	print com os argumentos da AST; comparações exibidas como verdadeiro ou falso
If	if e, quando presente, else
While	while
Break	break
Corpo vazio	pass

Todos os nomes recebem o prefixo `ms_var_`. O mapeamento é injetivo: nomes diferentes continuam diferentes. A variável `class`, válida na MemeScript, vira `ms_var_class`. A variável `print` vira `ms_var_print` e não sobrescreve a função de saída do Python. Literais de texto são emitidos com `repr`, sem interpretar seu conteúdo como código.

Exemplo de fonte:

```
E_HORA_DO_SHOW
SABOR_INTEIRO numero = 67;
SABOR_TEXTO mensagem = "Receba!";
RECEBA numero = numero + 1;
AMOSTRADINHO(mensagem, numero);
JA_ACABOU_JESSICA
```

Código gerado, omitindo os comentários de identificação:

```
ms_var_numero = 67
ms_var_mensagem = 'Receba!'
ms_var_numero = (ms_var_numero + 1)
print(ms_var_mensagem, ms_var_numero)
```

A saída é `Receba! 68`. A transpilação não executa o destino. A equipe pode examinar o arquivo gerado e executá-lo em uma etapa separada.

Programa completo de demonstração

O arquivo 02_valido_completo.meme inclui os três tipos, entrada/saída, if/else, while, interrupção e expressões cuja precedência altera o resultado. O código completo entregue é:

```
E_HORA_DO_SHOW
SABOR_INTEIRO energia = 0;
SABOR_INTEIRO cafes = 0;
SABOR_INTEIRO limite = 0;
SABOR_REAL preco = 4.50;
SABOR_REAL gasto = 0;
SABOR_TEXTO pedido = "QUERO CAFEEEEEEEE!";
AMOSTRADINHO("Qual o limite de cafés?");
QUERO_CAFE(limite);
BORA_BILL (cafes < limite)
    RECEBA cafes = cafes + 1;
    RECEBA energia = energia + 2 + 3 * 4;
    RECEBA gasto = cafes * preco;
    AMOSTRADINHO("Cafés:", cafes, "Energia:", energia);
    PODE_ISSO_ARNALDO (energia >= 67)
        AMOSTRADINHO("Já estou elétrico!");
        DESCANSAR_NE;
    ERROU
        AMOSTRADINHO(pedido);
ATA
AMOSTRADINHO("Energia final:", energia);
AMOSTRADINHO("Gasto:", gasto);
AMOSTRADINHO("Precedência:", 2 + 3 * 4);
AMOSTRADINHO("Parênteses:", (2 + 3) * 4);
JA_ACABOU_JESSICA
```

O exemplo no diretório exemplos também contém espaços e um comentário explicativo. Com entrada 10, o break ocorre no quinto café, quando a energia chega a 70. O gasto é 22.5. As duas últimas expressões produzem 14 e 20. Com limite 0, o laço não executa e o gasto permanece 0.0.

Suíte de testes e resultados

Além dos cinco casos mínimos exigidos, a entrega contém redeclaração, incompatibilidade, break inválido, aninhamento e entrada de tipos distintos. Os arquivos Python gerados dos quatro exemplos válidos estão no diretório gerados.

Arquivo no diretório exemplos	Resultado esperado
01_valido_basico.meme	Receba! 68
02_valido_completo.meme	Com entrada 10, energia 70 e gasto 22.5
03_erro_lexico.meme	Erro léxico, linha 3, coluna 24, símbolo @
04_erro_sintatico.meme	Erro sintático, linha 2, coluna 24, expressão ausente
05_erro_semantico.meme	Erro semântico, linha 2, coluna 8, variável não declarada
06_erro_redeclaracao.meme	Erro semântico, linha 3, coluna 12, nome duplicado
07_erro_tipos.meme	Erro semântico, linha 3, coluna 8, texto para inteiro
08_erro_break.meme	Erro semântico, linha 2, coluna 1, break fora de laço
09_valido_aninhado.meme	Break encerra apenas o laço interno
10_valido_tipos.meme	Com Pedro e 13.4, saída Pedro 67 6.7

Na validação desta versão, 32 métodos de unittest passaram no ambiente Linux com Python 3.13.16. Alguns métodos usam subcasos. A suíte verifica maior casamento, posições CRLF, limites de palavras reservadas, escapes, AST, precedência, associatividade, tipos, promoção, entrada, saída, nomes do Python e comportamento da CLI. A demonstração também executou os quatro programas válidos e rejeitou os seis inválidos.

O registro em resultados_validacao.txt contém a execução dos comandos. Os tempos observados nesse ambiente não são um benchmark nem uma medida de desempenho da linguagem.

Tratamento de erros

CompileError guarda categoria, mensagem, linha e coluna. Erros léxicos, sintáticos e semânticos encerram a compilação com código 1. Erros sintáticos informam o lexema encontrado e o esperado. A CLI grava o arquivo Python somente depois de concluir todas as análises; um erro preserva qualquer arquivo de saída anterior.

Execução e inspeção

Python 3.10 ou superior é o único pré-requisito. Não é necessário instalar bibliotecas. Execute os comandos na pasta do projeto. Em Linux, substitua python por python3 quando necessário; no Windows, py -3 também pode ser usado.

```
python -m memescript exemplos/02_valido_completo.meme -o gerados/02_valido_completo.py
python gerados/02_valido_completo.py
python -m unittest discover -s testes -v
python demo.py
python -m memescript exemplos/01_valido_basico.meme --check --tokens --ast --symbols
```

As opções de inspeção exibem JSON após a compilação bem-sucedida. A pasta gerados inclui também arquivos .tokens.json, .ast.json e .symbols.json dos programas válidos. --check apenas valida, sem gravar Python. A alternativa python transpilador.py usa a mesma implementação.

Limitações da versão

A linguagem não inclui funções, vetores, escopos locais, operadores lógicos compostos, conversões explícitas ou concatenação. Comentários são de linha. Comparações encadeadas são excluídas pela gramática. Essas restrições delimitam a implementação e correspondem aos arquivos entregues.

O código gerado depende da semântica numérica do Python. Entradas incompatíveis, divisão por zero e limites numéricos em execução não são erros estáticos de tipo. O transpilador não demonstra que um laço termina. Aninhamento muito profundo depende do limite de recursão do runtime.

Uso de inteligência artificial

Foram utilizados o ChatGPT (OpenAI), na proposta temática, no código, nos testes e nos materiais de apresentação, e o Claude (Anthropic), na revisão final do relatório, da apresentação e do código. Todo o material foi revisado e testado pelo grupo, que responde pela solução entregue.

Referências e correspondência com o enunciado

Fonte normativa. Cavalheiro, Marcos Ronaldo Melo. Trabalho Prático 1 — Projeto e Implementação de uma Linguagem de Programação Temática, versão 6. Documento da disciplina Linguagens Formais e Compiladores, UNIJUÍ, disponibilizado na disciplina.

Documentação técnica. Python Software Foundation. Python 3 Documentation: re, dataclasses, unittest, built-in functions e controle de fluxo. As páginas documentam as APIs usadas, não uma gramática externa copiada para a MemeScript.

<https://docs.python.org/3/library/re.html>

<https://docs.python.org/3/library/dataclasses.html>

<https://docs.python.org/3/library/unittest.html>

<https://docs.python.org/3/library/functions.html>

<https://docs.python.org/3/tutorial/controlflow.html>

Referências temáticas consultadas em 6 de outubro de 2026. É hora do show inspira a abertura do programa. Já acabou, Jéssica inspira o fechamento. Pode isso, Arnaldo inspira o teste de condição, e Errou, do Faustão, o ramo alternativo. A CNN reúne Receba e Bora Bill entre os virais de 2022. A Exame descreve o bordão Sabor energético. Os comandos técnicos e a família SABOR são adaptações próprias.

<https://33giga.com.br/e-hora-do-show-confira-memes-saidos-do-iconico-video-de-bambam-e-companhia>

<https://museudememes.com.br/collection/ja-acabou-jessica>

<https://redeglobo.globo.com/rj/tvriosul/Quem-Somos/noticia/pode-isso-arnaldo-novo-quadro-do-rj1-com-arnaldo-cezar-coelho.ghml>

<https://gshow.globo.com/tv/noticia/publico-se-diverte-revendo-quadro-do-domingao-que-deu-origem-ao-classico-meme-do-faustao-errou.ghml>

<https://www.cnnbrasil.com.br/pop/veja-os-memes-que-bombaram-nas-redes-sociais-em-2022/>

<https://exame.com/pop/sabor-energetico-entenda-o-meme-de-toguro-que-conquistou-ate-a-cimed/>

Requisito técnico	Evidência na entrega
Alfabeto e tabela de tokens	Especificação léxica deste relatório e lexer.py
$G = (V, T, P, S)$ e EBNF completa	Definição formal, produções e gramatica.ebnf
Léxico, parser e AST	lexer.py, parser.py e ast_nodes.py
Símbolos e tipos	semantic.py e casos inválidos
Geração equivalente	codegen.py e quatro programas gerados
Suíte válida e inválida	exemplos, testes e resultados_validacao.txt
Apresentação e defesa	docs/apresentacao.pptx e demonstração com demo.py